BON VOYAGE BOT

A Conversational AI Agent for Personalized Travel Planning

PROBLEM STATEMENT

Contemporary travel planning is a fragmented, time-intensive process that requires users to navigate multiple platforms including search engines, flight aggregators, hotel booking sites, and itinerary tools that neither share context nor adapt to individual user needs. Existing solutions produce generic recommendations that fail to account for personal budget constraints, travel preferences, or trip-specific requirements. Further, once a plan is generated, modifying it requires users to restart the process entirely, creating significant friction in the overall planning process.

TARGET USER

Bon Voyage Bot is designed for casual travelers planning domestic or international trips, regardless of group size or composition. The target user is someone who lacks the time for extensive manual research, finds the current landscape of travel tools overwhelming and disjointed, and desires a planning experience that is both adaptive and conversational.

GOAL

The goal of this project is to design and implement an end-to-end conversational AI travel planning assistant that enables users to describe their trip requirements in natural language and receive a personalized and budget-aware itinerary with the ability to refine their plan through dialogue. The system aims to consolidate the fragmented travel planning process into a single intelligent interface.

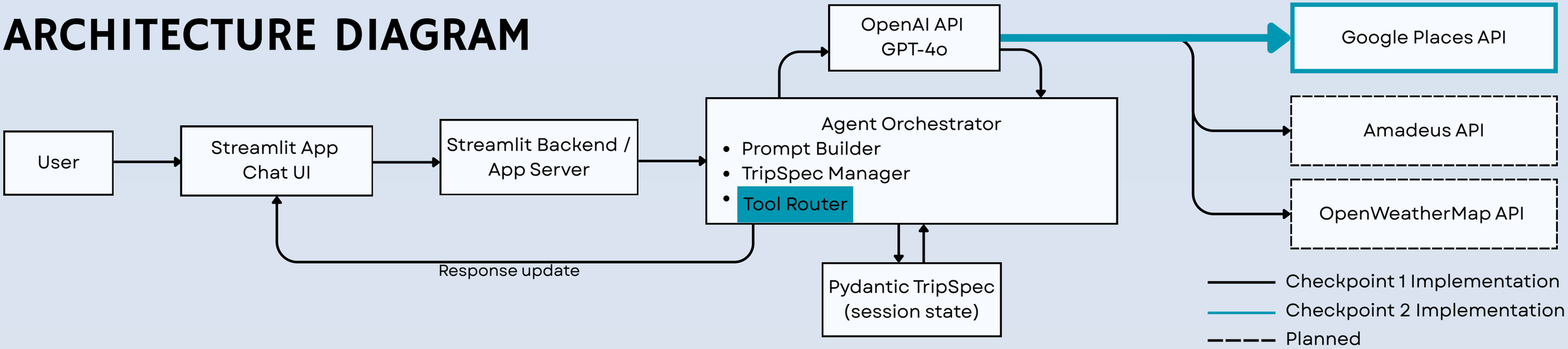
Since Checkpoint 1, the goal has been refined to emphasize an agentic architecture in which the system autonomously determines when and what to query from external APIs based on the state of the conversation. The intended final output of the system is a fully formatted, personalized day-by-day itinerary that consolidates real flight options, hotel recommendations, restaurants, and activities into a single cohesive travel plan.

METHODOLOGY

PROPOSED TECHNICAL APPROACH

Bon Voyage Bot is implemented as a conversational AI agent built on GPT-4o via the OpenAI API, deployed through a Streamlit web interface. The core system design centers on an agent orchestrator that manages prompt construction, tool routing, and structured state management. User inputs are processed through a dual pipeline: GPT-4o generates conversational responses while simultaneously extracting structured trip details into a Pydantic schema, TripSpec. The Tool Router evaluates TripSpec for completeness to determine when sufficient information exists to invoke external APIs including Amadeus for flight and hotel searches, Google Places for activity recommendations, and OpenWeatherMap for weather data. This architecture enables the system to ground LLM responses in real world data while maintaining a personalized conversation.

ARCHITECTURE DIAGRAM



Since Checkpoint 1, significant progress has been made in extending the system toward an agentic architecture. A Tool Router was implemented to evaluate the completeness of the TripSpec schema and determine which APIs have sufficient information to be invoked. The Google Places API was integrated using OpenAI function calling, allowing GPT-4o to autonomously determine when and what to search for based on the natural flow of conversation rather than triggering API calls automatically.

CODE SNIPPET I

bon-voyage-bot/agent/tool_router.py

```python
def get_required_tools(spec: TripSpec) -> list[str]:
    tools = []

    has_origin = spec.origin.city != ""
    has_destination = spec.destination.city != ""
    has_dates = spec.dates.nights != None
    has_budget = spec.budget.total_usd != None
    has_travelers = spec.travelers.adults > 0

    if has_destination: # Google Places API - requires destination
        tools.append("google_places")

    if has_destination and has_dates: # OpenWeatherMap API - requires destination, dates
        tools.append("weather")

    if has_origin and has_destination and has_dates and has_budget and has_travelers:
        # Amadeus API (flights) - requires origin, destination, dates, budget, travelers
        tools.append("flights")

    if has_destination and has_dates and has_budget: # Amadeus API (hotels) - requires dest, dates, budget
        tools.append("hotels")

    return tools
```

bon-voyage-bot/app/main.py

```python
# Check which tools are ready based on updated TripSpec
ready_tools = get_required_tools(st.session_state.trip_spec)

# Build active tools list based on what Tool Router says is ready
active_tools = [tool for tool in TOOLS if tool["function"]["name"] in ready_tools]

# Get chat response with active tools
with st.chat_message("assistant"):
    reply = get_chat_response(st.session_state.messages, active_tools)
```

EXPLANATION OF SNIPPET 1

FUNCTIONALITY

The code shown spans two files and together implements the tool routing pipeline. The get_required_tools() function in tool_router.py evaluates 5 conditions including origin city, destination city, dates, budget, and traveler count, and returns a list of APIs that have sufficient information to be invoked. Each API has its own completeness threshold reflecting the real-world requirements. For example, in order to search for flights, a user must know the origin, destination, dates, and number of travelers. The corresponding section of main.py then uses this list to dynamically build an active_tools list, ensuring GPT-4o only has access to APIs that are ready to return meaningful results.

IMPORTANCE

The Tool Router serves as the rule-based decision layer in the two-tier agentic architecture of Bon Voyage Bot. While OpenAI function calling allows GPT-4o to autonomously invoke APIs based on the conversational context (shown in code snippet 2), the Tool Router acts as a gatekeeper that ensures API calls are only made when the TripSpec contains sufficient structured data to return meaningful results. This design also allows for easy additions of new APIs, since adding a new API only requires adding a new completness check to the router.

Separating tool routing logic into its own dedicated file keeps the codebase modular and maintainable. Each condition maps directly to a real world API requirement, making the logic easy to understand, test, and extend as new services are integrates.

CODE SNIPPET 2

bon-voyage-bot/agent/chat.py

```
# Tool definition - tells GPT-4o what functions it can call
TOOLS = [{"type": "function", "function": {"name": "search_places",
    "description": "Search for restaurants, beaches, or attractions",
    "parameters": {"type": "object",
        "properties": {"query": {"type": "string"},
                       "destination": {"type": "string"}},
        "required": ["query", "destination"]}}}]

def get_chat_response(messages: list, tools: list = None) -> str:
    # Only pass tools enabled by Tool Router
    response = client.chat.completions.create(model="gpt-4o-mini", messages=messages,
                tools=tools if tools else None, tool_choice="auto" if tools else None)
    message = response.choices[0].message
    if message.tool_calls: # GPT-4o autonomously decided to call a tool
        tool_call = message.tool_calls[0]
        args = json.loads(tool_call.function.arguments)
        places = search_places(args["query"], args["destination
```

bon-voyage-bot/services/places.py

```
def search_places(query: str, destination: str) -> list[dict]:
    full_query = f"{query} in {destination}"
    headers = {"Content-Type": "application/json", "X-Goog-Api-Key": PLACES_API_KEY,
                "X-Goog-FieldMask": "places.displayName,places.shortFormattedAddress,places.rating,places.priceLevel"}
    payload = {"textQuery": full_query, "maxResultCount": 5}
    response = requests.post(PLACES_BASE_URL, json=payload, headers=headers)
```

EXPLANATION OF SNIPPET 2

FUNCTIONALITY

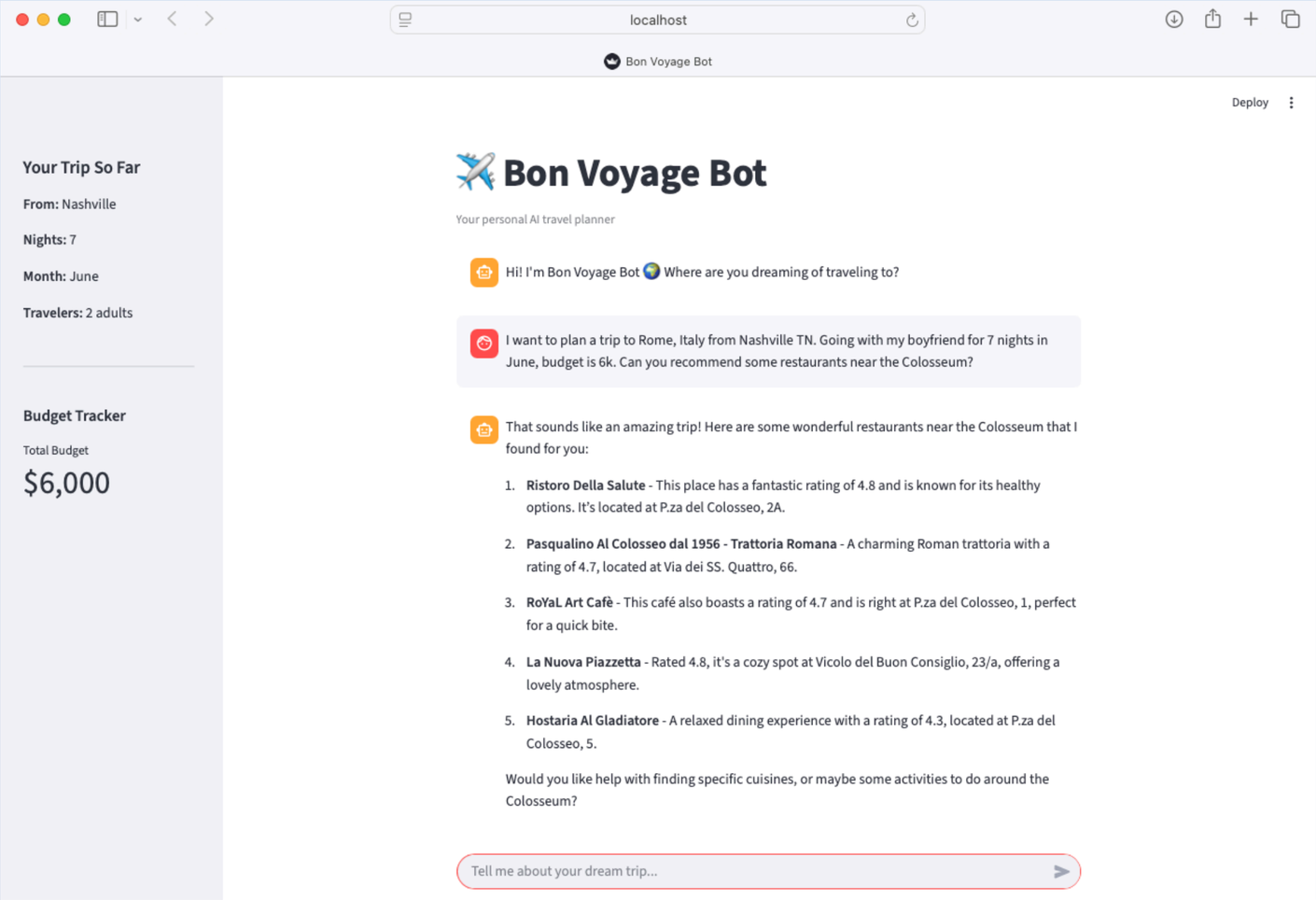
The code shown spans two files and together implement the function calling pipeline that connects the Tool Router from Snippet 1 to a real external data source. In chat.py, the TOOLS list defines the available functions in schemas that GPT-4o can understand, including the name, description, and required parameters. As of this checkpoint, the only available function if the search_places function that calls the Google Places API.

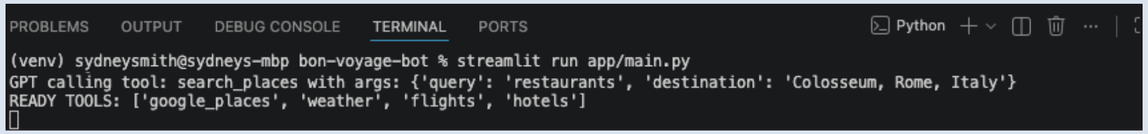
The get_chat_response() function receives the active_tools list built by the Tool Router from Snippet 1 and passes these definitions to GPT-4o only when sufficient trip context has been established. If GPT-4o autonomously decides to invoke the tool, the extracted arguments are passed to the search_places() function, which constructs a natural language query and submits it to the Google Places API, which returns up to five real location results, including name, address, rating, and price level.

IMPORTANCE

This snippet represents the most significant addition since Checkpoint 1. The two snippets work together, code block 1 determines which tools are eligible to be called based on the TripSpec completeness, while code block 2 defines what those tools do and how GPT-4o invokes them. This agentic nature is what sets Bon Voyage Bot apart from a basic chatbot and qualifies it as an AI agent, grounding its responses in real world location data rather than LLM training knowledge.

NEW PRELIMINARY RESULT





RESULT ANALYSIS & NEXT STEPS

ANALYSIS

The result demonstrates that the full agentic pipeline is functioning as intended. Upon receiving a single conversational message, the system extracted the structured trip details into the TripSpec schema, as shown in the sidebar containing origin, dates, travelers, and budget. It also enabled GPT-4o to autonomously invoke the Google Places API to retrieve five real restaurant recommendations near the Colosseum in Rome. This confirms that the Tool Router, function calling layer, and Places integration are all working correctly.

NEXT STEPS

- Expand function calling to support additional APIs including Amadeus for real flight and hotel search and OpenWeatherMap for destination weather
- Build an interactive itinerary builder where users can select restaurants, activities, and hotels to compile into a structured day-by-day trip plan
- Generate a formatted final itinerary as a downloadable PDF output
- Add photo and Google Maps link support to Places results for a richer user experience
- Deploy the application to Streamlit Community Cloud for public access